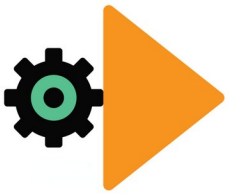

E-TOOLS AI CORPORATION

Abe Pro

Machine Learning Manual (Level 2)

Level 2: Tensors · Layers · Models · Attention · Inference · Training

Compiler: abec 1.0.0 (v4.1 grammar) · Linux x86-64 · Revised 2026-07-02

Contents

Overview

This is the **Level 2** companion to the *Abe Pro User Manual* (Level 1 + runtime). Level 1 covers general-purpose programming and the runtime library; **Level 2 covers machine learning** — the tensor type and operations, neural network `layer`s and `model`s, the modern transformer building blocks (attention, RoPE, RMSNorm, SwiGLU), autoregressive inference (KV cache, sampling, `generate`), training (`train`, optimizers, autograd, LoRA), and quantization (shrinking weights to int8 / 4-bit).

Everything here compiles **straight to native code** through `abec` and links the same compact runtime as Level 1 — there is no Python, no framework, no external tensor library to install. The runtime's tensor math is CPU `f64` today (a GPU backend exists but is optional); the focus is a correct, self-contained ML toolchain you can compile into a single binary.

> **Honesty contract.** Every code example in this manual is compiled **and run** by the regression suite (`tests/run_manual_ml_examples.sh`, `make test-manual-ml`) against the exact `abec` you download. If an example appears here, it compiles and produces the result shown. Where a construct is specified by the grammar but not yet lowered by the compiler, this manual says so explicitly rather than showing code that would not compile.
>
> **Prerequisites.** Read Level 1 first — Level 2 uses its functions, control flow, classes, and types throughout. Install `abec` + the runtime as described in Level 1, Chapter 1.

---

1. Tensors

A **tensor** is a multi-dimensional array of numbers — the value every ML computation flows through. In Abe a tensor has a static type written `Tensor<dtype, dims...>`, e.g. `Tensor<f64, 2, 3>` is a 2×3 matrix of 64-bit floats. The element type is `f64` in this release; the dimensions are part of the type so shapes are visible in your code.

1.1 Creating tensors

`zeros`, `ones`, `full`, and `randn` build tensors of a given shape. `tensor_sum` adds every element (handy for checking results), and `tensor_numel` returns the element count.

```
function main(): i64 {
    let a: Tensor<f64, 2, 3> = ones(2, 3);   // 6 elements, all 1.0
    if (tensor_sum(a) != 6.0) return 1;
    let z: Tensor<f64, 2, 3> = zeros(2, 3);  // all 0.0
    if (tensor_sum(z) != 0.0) return 2;
    return 0;
}
```

`full(rows, cols, value)` fills a shape with one value; `randn(rows, cols)` draws from a standard normal distribution:

```
function main(): i64 {
    let f = full(2, 2, -1.5);            // 4 elements, all -1.5
    if (tensor_sum(f) != -6.0) return 1;
    if (tensor_numel(f) != 4) return 2;  // element count
```

```
    let r = randn(3, 4);                // random-normal, shape [3,4]
    if (tensor_numel(r) != 12) return 3;
    return 0;
}
```

The type annotation is optional — `let a = ones(2, 3)` infers a tensor type — but annotating shapes documents intent and is recommended for layer/model code.

1.2 Element-wise arithmetic

`+`, `-`, `*`, and `/` operate **element-wise** on two same-shape tensors:

```
function main(): i64 {
    let a = full(2, 2, 6.0);
    let b = full(2, 2, 3.0);
    if (tensor_sum(a + b) != 36.0) return 1;   // 9.0 * 4
    if (tensor_sum(a - b) != 12.0) return 2;   // 3.0 * 4
    if (tensor_sum(a * b) != 72.0) return 3;   // 18.0 * 4 (element-wise)
    if (tensor_sum(a / b) != 8.0)  return 4;   // 2.0 * 4
    return 0;
}
```

Note that `*` is the **element-wise (Hadamard) product**, not matrix multiplication — for that, see `matmul` below.

1.3 Scaling by a scalar

Multiplying a tensor by a number scales every element. The scalar may be on either side:

```
function main(): i64 {
    let a = ones(2, 2);
    let c = a * 2.0;        // scale every element by a scalar
    if (tensor_sum(c) != 8.0) return 1;
    let d = 3.0 * a;        // scalar on either side
    if (tensor_sum(d) != 12.0) return 2;
    return 0;
}
```

Scaling is the only mixed tensor/scalar arithmetic: `+`, `-`, and `/` require two tensors of the same shape (add a `full(...)` tensor if you need to shift or divide by a constant).

1.4 Matrix multiplication

`matmul(a, b)` is the matrix product: an `[M, K]` tensor times a `[K, N]` tensor gives an `[M, N]` result.

```
function main(): i64 {
    let a: Tensor<f64, 2, 3> = ones(2, 3);
    let b: Tensor<f64, 3, 2> = ones(3, 2);
    let c: Tensor<f64, 2, 2> = matmul(a, b);  // each entry = 3.0, four entries
    if (tensor_sum(c) != 12.0) return 1;
    return 0;
}
```

1.5 Activations

The element-wise activations `relu`, `sigmoid`, and `tanh` are built in:

```
function main(): i64 {
    // relu clamps negatives to zero.
    let n = full(2, 2, -1.0);
    if (tensor_sum(relu(n)) != 0.0) return 1;
    // sigmoid(0) = 0.5 exactly, so a 4-element zero tensor sums to 2.0.
    let z = zeros(4, 1);
    if (tensor_sum(sigmoid(z)) != 2.0) return 2;
    // tanh(0) = 0.
    if (tensor_sum(tanh(z)) != 0.0) return 3;
    return 0;
}
```

The modern transformer activations `silu`, `swiglu`, `rmsnorm`, and `rope` are covered in Chapter 4 (Attention and transformer blocks).

1.6 Reshaping and inspecting

`reshape` reinterprets a tensor's elements under a new shape (the element count must match). `tensor_numel` and `tensor_sum` inspect a tensor, and `tensor_print` / `console.log(t)` dump it for debugging.

```
function main(): i64 {
    let a = ones(2, 6);             // 12 elements
    let b = reshape(a, 3, 4);       // same 12 elements, new shape
    if (tensor_numel(b) != 12) return 1;
    if (tensor_sum(b) != 12.0) return 2;   // values preserved
    return 0;
}
```

Function	Result
`zeros(r, c)` / `ones(r, c)`	a tensor filled with 0.0 / 1.0
`full(r, c, v)`	a tensor filled with `v`
`randn(r, c)`	standard-normal random values
`matmul(a, b)`	matrix product
`a + b` / `a - b` / `a * b` / `a / b`	element-wise (same shape)
`a * scalar`	scale every element
`relu(t)` / `sigmoid(t)` / `tanh(t)`	element-wise activations
`reshape(t, dims...)`	same data, new shape
`tensor_numel(t)`	element count (`i64`)
`tensor_sum(t)`	sum of all elements (`f64`)
`tensor_print(t)` / `console.log(t)`	print for debugging

---

2. Layers

A **layer** is a reusable neural-network building block: it bundles trainable **parameters** with a `forward` method that computes the layer's output. Layers are declared with `layer`, take compile-time **type parameters** (typically the input/output dimensions), instantiate with `new`, and run via `.forward(...)`.

2.1 A linear layer

This `Linear` layer holds a weight matrix and a bias vector, and its `forward` computes `weight · x + bias`. The type parameters `In`/`Out` flow into the parameter shapes:

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
function main(): i64 {
    let l = new Linear<8, 4>();
    let x: Tensor<f64, 8, 1> = ones(8, 1);
    let y = l.forward(x);          // matmul(ones[4,8], ones[8,1]) = 8 per output
    if (tensor_sum(y) != 32.0) return 1;   // 4 outputs * 8
    return 0;
}
```

The pieces:

- `param name: Type = expr`** declares a trainable parameter. The initializer `expr` runs when the layer is constructed, in scope of the type parameters — so `ones(Out, In)` allocates a correctly-shaped tensor. (Common initializers: `zeros`, `ones`, `full`, `randn`.)
- `forward(args): Ret { ... }`** is the layer's computation. Inside it, `this.weight` / `this.bias` access the parameters.
- `new Linear<8, 4>()`** constructs an instance with `In=8, Out=4`.
- `l.forward(x)`** runs it.

2.2 Activations and bias

A `forward` is ordinary Abe code over tensors, so you compose `matmul`, element-wise `+`, and activations freely:

```
layer Affine<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = full(Out, 1, -10.0);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return relu(matmul(this.weight, x) + this.bias);
    }
}
function main(): i64 {
    let l = new Affine<8, 4>();
    let x: Tensor<f64, 8, 1> = ones(8, 1);
```

```
    let y = l.forward(x);          // 8 + (-10) = -2 per output, relu -> 0
    if (tensor_sum(y) != 0.0) return 1;
    return 0;
}
```

2.3 Buffers — persistent, non-trainable state

A **`buffer`** is like a `param` but is **not** updated by the optimizer: use it for fixed state a layer needs at inference time (scales, masks, running statistics). It is declared and accessed exactly like a parameter:

```
layer Scaler<N> {
    param  weight: Tensor<f64, N, N> = ones(N, N);      // trainable
    buffer factor: Tensor<f64, N, 1> = full(N, 1, 2.0); // persistent, not trained
    forward(x: Tensor<f64, N, 1>): Tensor<f64, N, 1> {
        return matmul(this.weight, x) * this.factor;
    }
}
function main(): i64 {
    let s = new Scaler<3>();
    let x: Tensor<f64, 3, 1> = ones(3, 1);
    let y = s.forward(x);          // matmul = 3 per row, * factor 2.0 = 6
    if (tensor_sum(y) != 18.0) return 1;   // 3 rows * 6
    return 0;
}
```

Shape convention. The runtime's `matmul` is 2-D, so layer inputs are written as column tensors `Tensor<f64, In, 1>` rather than 1-D vectors. This keeps every `forward` a sequence of well-defined matrix operations.

A layer may also declare **custom methods** beyond `forward` — they dispatch as `instance.method(args)` and can be called from `forward` via `this.method(args)` — and a custom **`init()` body**, which runs at construction (after `param = expr` initializers) with `this` in scope, so it can read params, call methods, and assign `this.field = expr`.

---

3. Models

A **model** composes layers (and other models) into a complete network. It is declared with `model`, holds **submodules** as fields, and — like a layer — has a `forward` method. Submodules are constructed automatically when the model is, so you never wire up children by hand.

3.1 Composing layers

This `MLP` holds two `Linear` submodules and threads its input through both. The submodule fields (`l1`, `l2`) are typed with concrete dimensions, and `this.l1.forward(...)` dispatches into each child:

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
```

```
        return matmul(this.weight, x) + this.bias;
    }
}
model MLP {
    l1: Linear<8, 4>;
    l2: Linear<4, 2>;
    forward(x: Tensor<f64, 8, 1>): Tensor<f64, 2, 1> {
        return this.l2.forward(this.l1.forward(x));
    }
}
function main(): i64 {
    let m = new MLP();
    let x: Tensor<f64, 8, 1> = ones(8, 1);
    let y = m.forward(x);          // l1: ones->8 per row; l2: 8*4 = 32 per row
    if (tensor_sum(y) != 64.0) return 1;   // 2 outputs * 32
    return 0;
}
```

`new MLP()` constructs the model and every submodule it owns; `m.forward(x)` runs the whole network.

3.2 Running a model with `infer`

`m.forward(x)` calls the model directly. You can also run it through an `infer` statement inside an `ai { }` block — the explicit-inference form that marks an inference boundary (and is the entry point for the inference options in later chapters):

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
model MLP {
    l1: Linear<8, 4>;
    l2: Linear<4, 2>;
    forward(x: Tensor<f64, 8, 1>): Tensor<f64, 2, 1> {
        return this.l2.forward(this.l1.forward(x));
    }
}
function main(): i64 {
    let m = new MLP();
    let x: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        let y = infer m(x);        // run the model inside an ai {} block
        if (tensor_sum(y) != 64.0) { r = 1; }
    }
    return r;
}
```

`infer m(x)` invokes the model's `forward`. (`ai { }` blocks are required around `infer`; they also scope the AI/generation operations covered later.)

3.3 Arrays of submodules

Real networks stack many identical blocks. Rather than declaring `block0`, `block1`, … by hand, declare an **array submodule** with `name: [N] T`. The model constructs all `N` children, and you dispatch with `this.name[i].forward`:

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
model Stack {
    blocks: [3] Linear<4, 4>;       // an array of 3 submodules
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {
        let h0 = this.blocks[0].forward(x);
        let h1 = this.blocks[1].forward(h0);
        let h2 = this.blocks[2].forward(h1);
        return h2;
    }
}
function main(): i64 {
    let s = new Stack();
    let x: Tensor<f64, 4, 1> = ones(4, 1);
    let y = s.forward(x);          // 4 -> 16 -> 64 per row
    if (tensor_sum(y) != 256.0) return 1;  // 4 rows * 64
    return 0;
}
```

The index may be a **runtime variable**, so you can loop over the stack — this is exactly how an N-layer transformer is applied:

```
layer Linear<In, Out> {
    param weight: Tensor<f64, Out, In> = ones(Out, In);
    param bias:   Tensor<f64, Out, 1>  = zeros(Out, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Out, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
model Stack {
    blocks: [3] Linear<4, 4>;
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {
        let i: i64 = 0;
        let h: Tensor<f64, 4, 1> = x;
        while (i < 3) {            // a variable index works the same way
            h = this.blocks[i].forward(h);
            i = i + 1;
```

```
        }
        return h;
    }
}
function main(): i64 {
    let s = new Stack();
    let x: Tensor<f64, 4, 1> = ones(4, 1);
    let y = s.forward(x);
    if (tensor_sum(y) != 256.0) return 1;
    return 0;
}
```

> **Loading real weights.** The models here initialize their parameters from `param = expr` initializers (`ones`, `randn`, ...). To load *pretrained* weights from disk — a `model_load(...)` per parameter, backed by a safetensors directory — see Chapter 8 (Loading models). The architecture wiring is identical; only the initializers change.

---

4. Attention and transformer blocks

This chapter covers the kernels every modern transformer (Llama, Mistral, Qwen, TinyLlama) is built from — `rmsnorm`, `silu`/`swiglu`, `rope`, and `gqa_attention` — and assembles them into one transformer block. They are built-in functions, so a block is plain Abe code over tensors.

4.1 RMSNorm

`rmsnorm(x, weight, eps)` normalizes `x` by the root-mean-square of its last axis and scales by a per-feature `weight` (a rank-1 tensor). It is the normalization used throughout Llama-family models.

```
function main(): i64 {
    // rmsnorm(x, weight, eps): x / sqrt(mean(x^2)+eps) * weight, on the last axis.
    // For x = ones and weight = ones with eps = 0, mean(x^2)=1 so the output is x.
    let x = ones(1, 4);
    let w = ones(4);                // per-feature scale, rank-1 [4]
    let n = rmsnorm(x, w, 0.0);
    if (tensor_sum(n) != 4.0) return 1;
    return 0;
}
```

4.2 SiLU and SwiGLU

`silu(x) = x · sigmoid(x)` is the activation in the feed-forward network, and `swiglu(gate, up) = silu(gate) · up` is the gated form Llama uses:

```
function main(): i64 {
    // silu(x) = x * sigmoid(x); silu(0) = 0.
    let z = zeros(8, 1);
    if (tensor_sum(silu(z)) != 0.0) return 1;
    // silu(1) = 0.731058...; four of them sum to ~2.9243.
    let s = tensor_sum(silu(ones(4, 1)));
```

```
    if (s < 2.92 || s > 2.93) return 2;
    // swiglu(gate, up) = silu(gate) * up; with both ones, == silu(1) * 1.
    let sw = tensor_sum(swiglu(ones(4, 1), ones(4, 1)));
    if (sw < 2.92 || sw > 2.93) return 3;
    return 0;
}
```

Irrational results like `silu(1) = 0.7310…` can't be checked with `==`, so the examples assert a tolerance band (`s < 2.92 || s > 2.93`) — the same technique you'll use to test your own models.

4.3 Rotary position embeddings (RoPE)

`rope(x, start_pos, theta_base)` applies rotary position encoding to a tensor shaped `[seq, heads, head_dim]`. Query and key projections are reshaped into heads, then rotated, before attention. At position 0 the rotation is the identity:

```
function main(): i64 {
    // rope(x, start_pos, theta_base) — rotary position embedding on
    // [seq, heads, head_dim]. At position 0 the rotation is the identity.
    let x = ones(1, 1, 4);
    let r = rope(x, 0, 10000.0);
    if (tensor_sum(r) != 4.0) return 1;
    // It also applies after reshaping a projection into heads:
    let q = ones(1, 8);
    let qm = reshape(q, 1, 4, 2);    // [seq=1, heads=4, head_dim=2]
    if (tensor_sum(rope(qm, 0, 10000.0)) != 8.0) return 2;
    return 0;
}
```

4.4 Grouped-query attention

`gqa_attention(q, k, v)` is grouped-query attention: several query heads share each key/value head (the memory-saving scheme TinyLlama and Llama-3 use). The inputs are shaped `[seq, heads, head_dim]`, with more query heads than KV heads. `tensor_set_flat(t, i, v)` writes element `i` in row-major order — handy for building a tensor with known values:

```
function main(): i64 {
    // 4 query heads, 2 KV heads, head_dim 2, single decode position.
    // Heads 0,1 read KV head 0; heads 2,3 read KV head 1. With one
    // position the softmax is 1.0, so each query head's output is its
    // KV head's value row.
    let q = full(1, 4, 2, 0.0);
    tensor_set_flat(q, 0, 1.0); tensor_set_flat(q, 2, 1.0);   // q heads 0,1
    tensor_set_flat(q, 5, 1.0); tensor_set_flat(q, 7, 1.0);   // q heads 2,3
    let k = full(1, 2, 2, 0.0);
    tensor_set_flat(k, 0, 1.0); tensor_set_flat(k, 2, 1.0);
    let v = full(1, 2, 2, 0.0);
    tensor_set_flat(v, 0, 2.0); tensor_set_flat(v, 1, 3.0);   // KV0 = [2,3]
    tensor_set_flat(v, 2, 4.0); tensor_set_flat(v, 3, 5.0);   // KV1 = [4,5]
    let o = gqa_attention(q, k, v);
    // heads 0,1 -> [2,3], heads 2,3 -> [4,5]: 2*(2+3) + 2*(4+5) = 28
```

```
    if (tensor_sum(o) != 28.0) return 1;
    return 0;
}
```

4.5 A transformer block

Putting it together: a transformer block is **pre-norm attention with a residual, then a pre-norm SwiGLU feed-forward with a residual**. Here the block is a function taking its weights as arguments (Chapter 8 shows how a `model` loads those weights from disk); hidden size 8, 4 query / 2 KV heads, head_dim 2, intermediate size 16:

```
// A complete transformer block: pre-norm attention (RoPE + grouped-query
// attention) with a residual, then a pre-norm SwiGLU feed-forward with a
// residual. Hidden = 8, 4 query heads / 2 KV heads, head_dim = 2,
// intermediate = 16. Weights are passed in (Chapter 8 loads them from disk).
function block(x:         Tensor<f64, 1, 8>,
               attn_norm: Tensor<f64, 8>,
               Wq: Tensor<f64, 8, 8>, Wk: Tensor<f64, 8, 4>,
               Wv: Tensor<f64, 8, 4>, Wo: Tensor<f64, 8, 8>,
               ffn_norm:  Tensor<f64, 8>,
               Wgate: Tensor<f64, 8, 16>, Wup: Tensor<f64, 8, 16>,
               Wdown: Tensor<f64, 16, 8>): Tensor<f64, 1, 8> {
    let h  = rmsnorm(x, attn_norm, 0.0);
    let q  = matmul(h, Wq);
    let k  = matmul(h, Wk);
    let v  = matmul(h, Wv);
    let qr = rope(reshape(q, 1, 4, 2), 0, 10000.0);
    let kr = rope(reshape(k, 1, 2, 2), 0, 10000.0);
    let am = gqa_attention(qr, kr, reshape(v, 1, 2, 2));
    let ao = matmul(reshape(am, 1, 8), Wo);
    let x1 = x + ao;                       // attention residual
    let hn = rmsnorm(x1, ffn_norm, 0.0);
    let f  = swiglu(matmul(hn, Wgate), matmul(hn, Wup));
    let mo = matmul(f, Wdown);
    return x1 + mo;                        // feed-forward residual
}
function main(): i64 {
    let x  = full(1, 8, 0.5);
    let an = full(8, 1.0);
    let y = block(x, an,
        full(8, 8, 0.125), full(8, 4, 0.125), full(8, 4, 0.125), full(8, 8, 0.125),
        an, full(8, 16, 0.125), full(8, 16, 0.5), full(16, 8, 0.0625));
    let s = tensor_sum(y);
    if (s < 35.39 || s > 35.40) return 1;  // closed-form ~35.3939
    return 0;
}
```

Stack this block `N` times (Chapter 3's array submodules), add an embedding lookup at the input and a final RMSNorm + projection at the output, and you have a complete language model — exactly what Chapter 8 loads real weights into.

Function	Purpose
`rmsnorm(x, weight, eps)`	RMS normalization over the last axis
`silu(x)`	`x * sigmoid(x)` activation
`swiglu(gate, up)`	`silu(gate) * up` gated FFN activation
`rope(x, start_pos, theta)`	rotary position embedding on `[seq, heads, head_dim]`
`gqa_attention(q, k, v)`	grouped-query attention
`tensor_set_flat(t, i, v)`	write element `i` (row-major) of a tensor

---

5. Inference and generation

Once a model can run a `forward`, you can **generate** text from it. This chapter covers the KV cache (which makes autoregressive decoding fast), the `generate` statement, and the lower-level `sample` for hand-written decode loops. Generation happens inside an `ai { }` block — the explicit boundary for inference operations.

5.1 The KV cache

A **KV cache** stores the key/value tensors from previous positions so each new token only computes attention against the cache instead of re-reading the whole sequence. Allocate one with `KVCache<dtype, layers, max_seq, head_dim>`; `kv_cache_seq_len` reports how many positions it holds:

```
function main(): i64 {
    let r: i64 = 0;
    ai {
        // KVCache<dtype, num_layers, max_seq, head_dim>
        let cache = KVCache<f64, 2, 16, 4>(maxBatchSize: 1);
        if (kv_cache_seq_len(cache) != 0) { r = 1; }   // empty to start
    }
    return r;
}
```

5.2 Generating tokens

`generate model(prompt) with { … }` runs an autoregressive decode loop: it calls the model's `forward`, samples a token, feeds it back, and repeats up to `maxNewTokens`. `temperature: 0.0` selects greedy decoding (argmax). The result is a tensor of token ids.

```
layer Tiny<In, Vocab> {
    param weight: Tensor<f64, Vocab, In> = ones(Vocab, In);
    param bias:   Tensor<f64, Vocab, 1>  = zeros(Vocab, 1);
    forward(x: Tensor<f64, In, 1>): Tensor<f64, Vocab, 1> {
        return matmul(this.weight, x) + this.bias;
    }
}
function main(): i64 {
    let lm = new Tiny<8, 5>();
```

```
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        // Greedy generation (temperature 0 = argmax). Uniform logits -> token 0.
        let toks = generate lm(prompt) with {
            maxNewTokens: 5;
            temperature:  0.0;
        };
        if (tensor_numel(toks) != 5) { r = 1; }   // five new tokens
        if (tensor_sum(toks) != 0.0) { r = 2; }   // all token id 0
    }
    return r;
}
```

5.3 Generating with a KV cache

For a cache-aware model, pass the cache to `generate` with `cache:`. The model's `forward` takes the cache as a parameter and appends each step's key/value with `kv_cache_append` / `kv_cache_advance`:

```
layer Cached {
    param weight: Tensor<f64, 5, 8> = ones(5, 8);
    forward(x: Tensor<f64, 8, 1>,
            cache: KVCache<f64, 1, 16, 4>): Tensor<f64, 5, 1> {
        // A real layer projects K/V here; this commits one position per step.
        kv_cache_append(cache, 0, ones(4, 1), ones(4, 1));
        kv_cache_advance(cache);
        return matmul(this.weight, x);
    }
}
function main(): i64 {
    let lm = new Cached();
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        let cache = KVCache<f64, 1, 16, 4>(maxBatchSize: 1);
        let toks = generate lm(prompt) with {
            maxNewTokens: 3;
            temperature:  0.0;
            cache:        cache;       // thread the cache through the decode loop
        };
        if (tensor_numel(toks) != 3) { r = 1; }
        if (kv_cache_seq_len(cache) != 3) { r = 2; }   // 3 positions cached
    }
    return r;
}
```

5.4 Sampling and manual decode loops

`generate` is convenient, but you can drive decoding yourself for full control. `sample(logits, temperature, topK, topP, seed)` turns a logits tensor into a token id — greedy when `temperature` is 0, otherwise temperature/top-k/top-p sampling:

```
function main(): i64 {
    // sample(logits, temperature, topK, topP, seed) -> token id.
    // Temperature 0 is greedy argmax; uniform logits pick index 0.
    let logits = ones(5, 1);
    let tok = sample(logits, 0.0, 0, 1.0, 0);
    if (tok != 0) return 1;
    return 0;
}
```

Putting it in a loop — call `forward`, `sample`, repeat — is exactly what `generate` does under the hood:

```
layer Cached {
    param weight: Tensor<f64, 5, 8> = ones(5, 8);
    forward(x: Tensor<f64, 8, 1>,
            cache: KVCache<f64, 1, 16, 4>): Tensor<f64, 5, 1> {
        kv_cache_append(cache, 0, ones(4, 1), ones(4, 1));
        kv_cache_advance(cache);
        return matmul(this.weight, x);
    }
}
function main(): i64 {
    let lm = new Cached();
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        // The lower-level alternative to `generate`: drive the loop yourself,
        // calling forward and sampling each step.
        let cache = KVCache<f64, 1, 16, 4>(maxBatchSize: 1);
        let i: i64 = 0;
        while (i < 3) {
            let logits: Tensor<f64, 5, 1> = lm.forward(prompt, cache);
            let tok: i64 = sample(logits, 0.0, 0, 1.0, 0);
            if (tok != 0) { r = 1; }
            i = i + 1;
        }
        if (kv_cache_seq_len(cache) != 3) { r = 2; }
    }
    return r;
}
```

Function / form	Purpose
`KVCache<dt, layers, max_seq, head_dim>(maxBatchSize: n)`	allocate a KV cache
`kv_cache_seq_len(c)`	positions currently cached (`i64`)
`kv_cache_append(c, layer, k, v)` / `kv_cache_advance(c)`	commit a position
`generate m(prompt) with { maxNewTokens; temperature; cache; }`	autoregressive decode

Function / form	Purpose
`sample(logits, temp, topK, topP, seed)`	logits → token id

---

6. Tokenization

A language model works on **token ids**, not text. A **tokenizer** converts between the two: `encode` turns a string into a tensor of ids, and `decode` turns ids back into a string. Abe's runtime has a byte-level **BPE** tokenizer (the scheme Llama, GPT-2, and friends use), loaded from a `tokenizer.json`.

6.1 Encoding and decoding

`tokenizer_load(path)` activates a tokenizer (returns 0 on success); `vocab_size()` reports the vocabulary size; `encode` / `decode` round-trip text; `decode_one(id)` decodes a single id; and `tokenizer_unload()` releases it. The example writes a tiny `tokenizer.json` inline so it is self-contained — a real program points `tokenizer_load` at a model's own tokenizer file.

```
function main(): i64 {
    // A real program loads a model's tokenizer.json; this writes a tiny
    // byte-level BPE fixture inline so the example is self-contained.
    mkdir("/tmp/abe_ml6_demo");
    write_text("/tmp/abe_ml6_demo/tokenizer.json",
        "{\"model\":{\"type\":\"BPE\",\"vocab\":
{\"h\":0,\"e\":1,\"l\":2,\"o\":3,\"ll\":4,\"lo\":5,\"he\":6,\"hell\":7,\"hello\":8,\
"\\u0120\":9,\"w\":10,\"r\":11,\"d\":12,\"o\\u0120\":13},\"merges\":[\"o \\
u0120\",\"l l\",\"l o\",\"h e\",\"he ll\",\"hell o\"]}}");

    let r: i64 = 0;
    if (tokenizer_load("/tmp/abe_ml6_demo/tokenizer.json") != 0) { r = 1; }
    if (vocab_size() != 14) { r = 2; }

    let ids = encode("hello");          // BPE merges all the way to token 8
    if (tensor_sum(ids) != 8.0) { r = 3; }
    if (decode(ids) != "hello") { r = 4; }   // round-trips back to text
    if (decode_one(6) != "he") { r = 5; }    // a single id -> its token

    tokenizer_unload();
    return r;
}
```

`encode` returns an ordinary tensor of ids, so it feeds straight into a model's `forward` / `generate`; `decode` takes the generated id tensor back to text.

6.2 Word boundaries

Byte-level BPE **pre-tokenizes** on word boundaries before merging, so a merge never spans two words. Encoding `"hello world"` keeps `hello` intact and starts a fresh run at the space — it does not merge the trailing `o` of *hello* with the leading space of *world*:

```
function main(): i64 {
```

```
    mkdir("/tmp/abe_ml6_demo2");
    write_text("/tmp/abe_ml6_demo2/tokenizer.json",
        "{\"model\":{\"type\":\"BPE\",\"vocab\":
{\"h\":0,\"e\":1,\"l\":2,\"o\":3,\"ll\":4,\"lo\":5,\"he\":6,\"hell\":7,\"hello\":8,\
"\\u0120\":9,\"w\":10,\"r\":11,\"d\":12,\"o\\u0120\":13},\"merges\":[\"o \\
u0120\",\"l l\",\"l o\",\"h e\",\"he ll\",\"hell o\"]}}");

    let r: i64 = 0;
    tokenizer_load("/tmp/abe_ml6_demo2/tokenizer.json");
    // Words are split before BPE, so merges never cross a word boundary:
    // "hello world" -> [hello=8, " "=9, w=10, o=3, r=11, l=2, d=12] = 55.
    let ids = encode("hello world");
    if (tensor_sum(ids) != 55.0) { r = 1; }
    if (decode(ids) != "hello world") { r = 2; }
    tokenizer_unload();
    return r;
}
```

The runtime tokenizer handles the full GPT-2 byte ↔ unicode mapping and UTF-8, so it round-trips multilingual text, not just ASCII.

Function	Purpose
`tokenizer_load(path)`	activate a `tokenizer.json` (returns 0 on success)
`vocab_size()`	vocabulary size (`i64`)
`encode(text)`	text → tensor of token ids
`decode(ids)`	id tensor → string
`decode_one(id)`	a single id → its token string
`tokenizer_unload()`	release the active tokenizer

---

7. Serving

Generating for one prompt is Chapter 5; **serving** runs many requests efficiently. This chapter covers the request **scheduler** (continuous batching), the **paged** KV cache and fused `paged_attention`, memory budgeting with page pools, and reproducible (deterministic) serving.

> **Generic-type spacing.** Write a space before `=` when a type ends in `>`: `let c: KVCache<f64, 1, 16, 4> = …`, not `…4>=…`. Without the space the lexer reads `>=` as a single token. All examples here follow this convention.

7.1 The scheduler

`scheduler_new(policy, maxBatch, maxPending)` creates a request scheduler (policy 0 = FIFO). You `submit` requests (each with its own KV cache and token budget), then loop: `pick` a ready request, run one `forward` step on its cache, `sample`, and `record` the token. `pending` drives the loop; `num_finished` and `get_tokens` read the results.

```
layer Cached {
```

```
    param weight: Tensor<f64, 5, 8> = ones(5, 8);
    forward(x: Tensor<f64, 8, 1>,
            cache: KVCache<f64, 1, 16, 4>): Tensor<f64, 5, 1> {
        kv_cache_append(cache, 0, ones(4, 1), ones(4, 1));
        kv_cache_advance(cache);
        return matmul(this.weight, x);
    }
}
function main(): i64 {
    let lm = new Cached();
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        let ca = KVCache<f64, 1, 16, 4>(maxBatchSize: 1);
        let cb = KVCache<f64, 1, 16, 4>(maxBatchSize: 1);
        // scheduler_new(policy, maxBatch, maxPending): policy 0 = FIFO.
        let sched = scheduler_new(0, 4, 8);
        let ra = scheduler_submit(sched, ca, 2);   // request A: 2 new tokens
        let rb = scheduler_submit(sched, cb, 3);   // request B: 3 new tokens
        // Drive the decode loop: pick a request, run one step, record its token.
        while (scheduler_pending(sched) > 0) {
            let req = scheduler_pick(sched);
            if (req < 0) { break; }
            let logits = lm.forward(prompt, scheduler_get_cache(sched, req));
            scheduler_record(sched, req, sample(logits, 0.0, 0, 1.0, 0));
        }
        if (scheduler_num_finished(sched) != 2) { r = 1; }
        if (tensor_numel(scheduler_get_tokens(sched, ra)) != 2) { r = 2; }
        if (tensor_numel(scheduler_get_tokens(sched, rb)) != 3) { r = 3; }
    }
    return r;
}
```

7.2 Paged attention

A **paged** KV cache stores tokens in fixed-size pages, allocated lazily as the sequence grows — the layout that lets a server pack many variable-length sequences into one pool. Enable it with `pagedKV: true, blockSize: n`. `paged_attention(cache, layer, q)` is a single fused op that walks the page table directly instead of gathering a contiguous K/V tensor:

```
layer Attn {
    param Wq: Tensor<f64, 4, 4> = ones(4, 4);
    param Wk: Tensor<f64, 4, 4> = ones(4, 4);
    param Wv: Tensor<f64, 4, 4> = ones(4, 4);
    param Wo: Tensor<f64, 5, 4> = ones(5, 4);
    forward(x: Tensor<f64, 4, 1>,
            cache: KVCache<f64, 1, 16, 4>): Tensor<f64, 5, 1> {
        kv_cache_append(cache, 0, matmul(this.Wk, x), matmul(this.Wv, x));
        kv_cache_advance(cache);
        // One fused op walks the paged cache — no contiguous K/V is built.
        let attn_t = paged_attention(cache, 0, matmul(this.Wq, x));
```

```
        return matmul(this.Wo, attn_t);
    }
}
function main(): i64 {
    let lm = new Attn();
    let prompt: Tensor<f64, 4, 1> = ones(4, 1);
    let r: i64 = 0;
    ai {
        // A paged cache stores tokens in fixed-size pages, allocated lazily.
        let cache = KVCache<f64, 1, 16, 4>(maxBatchSize: 1, pagedKV: true,
blockSize: 2);
        let toks = generate lm(prompt) with { maxNewTokens: 4; temperature: 0.0;
cache: cache; };
        if (tensor_numel(toks) != 4) { r = 1; }
        if (kv_cache_seq_len(cache) != 4) { r = 2; }
    }
    return r;
}
```

7.3 Page pools and variable-length serving

`poolPages` budgets the cache's memory **below** the worst case. When the pool is exhausted, you reclaim a finished sequence's pages with `kv_cache_release`, and the freed pages serve the next request — the substrate for packing many variable-length sequences into bounded memory:

```
function main(): i64 {
    let r: i64 = 0;
    ai {
        // poolPages budgets memory below the worst case (4 pages here): only
        // 2 pages exist, so a long sequence must release pages between rounds.
        let cache = KVCache<f64, 1, 8, 4>(maxBatchSize: 1, pagedKV: true, blockSize:
2, poolPages: 2);
        let i: i64 = 0;
        while (i < 4) { kv_cache_append(cache, 0, ones(4,1), ones(4,1));
kv_cache_advance(cache); i = i + 1; }
        if (kv_cache_seq_len(cache) != 4) { r = 1; }
        kv_cache_release(cache, 0);              // return this row's pages to the
pool
        if (kv_cache_seq_len(cache) != 0) { r = 2; }
        let j: i64 = 0;
        while (j < 4) { kv_cache_append(cache, 0, ones(4,1), ones(4,1));
kv_cache_advance(cache); j = j + 1; }
        if (kv_cache_seq_len(cache) != 4) { r = 3; }   // pages reused for the next
round
    }
    return r;
}
```

7.4 Deterministic serving

For reproducible output (tests, debugging, audits), `set_deterministic(seed)` fixes the sampling RNG so the same seed and config produce identical tokens:

```
layer Tiny {
    param weight: Tensor<f64, 5, 8> = ones(5, 8);
    forward(x: Tensor<f64, 8, 1>): Tensor<f64, 5, 1> { return matmul(this.weight,
x); }
}
function main(): i64 {
    let lm = new Tiny();
    let prompt: Tensor<f64, 8, 1> = ones(8, 1);
    let r: i64 = 0;
    ai {
        // Same seed + config => identical sampled output (reproducible serving).
        set_deterministic(42);
        let a = generate lm(prompt) with { maxNewTokens: 4; temperature: 1.0; topK:
5; seed: 0; };
        set_deterministic(42);
        let b = generate lm(prompt) with { maxNewTokens: 4; temperature: 1.0; topK:
5; seed: 0; };
        if (tensor_sum(a) != tensor_sum(b)) { r = 1; }
        if (tensor_numel(a) != 4) { r = 2; }
    }
    return r;
}
```

The runtime also exposes a telemetry log (`telemetry_clear` / `telemetry_emit` / `telemetry_count` / `telemetry_dump`) for recording per-request events.

Function	Purpose
`scheduler_new(policy, maxBatch, maxPending)`	create a scheduler (0 = FIFO)
`scheduler_submit(s, cache, maxNew)`	enqueue a request → request id
`scheduler_pending(s)` / `scheduler_pick(s)`	loop control / next ready request
`scheduler_get_cache(s, req)`	the request's KV cache
`scheduler_record(s, req, tok)`	append a sampled token
`scheduler_num_finished(s)` / `scheduler_get_tokens(s, req)`	results
`KVCache<…>(…, pagedKV: true, blockSize: n, poolPages: p)`	paged cache + pool budget
`paged_attention(cache, layer, q)`	fused page-aware attention
`kv_cache_release(cache, row)`	return a row's pages to the pool
`set_deterministic(seed)`	fix the sampling RNG

---

8. Loading models

Every model so far initialized its parameters from `ones`/`randn`/`full`. This chapter loads **real pretrained weights** from disk in the **safetensors** format — the same files HuggingFace ships — so the architectures from Chapters 2–4 run with actual weights.

8.1 Saving and loading tensors

`save_safetensors(path, name, tensor)` writes a tensor; `load_safetensors(path, name)` reads it back. A layer loads its weight by calling `load_safetensors` in the parameter initializer — it runs when the layer is constructed, so the parameter holds the on-disk tensor by the time `new` returns:

```
layer Loaded {
    // The initializer reads the weight from disk when the layer is constructed.
    param weight: Tensor<f64, 5, 8> =
        load_safetensors("/tmp/abe_ml8_w.safetensors", "weight");
    forward(x: Tensor<f64, 8, 1>): Tensor<f64, 5, 1> {
        return matmul(this.weight, x);
    }
}
function main(): i64 {
    let w = full(5, 8, 0.5);
    save_safetensors("/tmp/abe_ml8_w.safetensors", "weight", w);
    let lm = new Loaded();             // __init reads the weight back
    let y = lm.forward(ones(8, 1));
    if (tensor_sum(y) != 20.0) return 1;   // 5 rows * (8 * 0.5) = 20
    return 0;
}
```

8.2 Model directories

Real LLMs ship as a **directory**: one or more `*.safetensors` shards, a `model.safetensors.index.json` mapping each tensor name to its shard, and a `config.json`. `model_open(dir)` opens it; `model_config_int(key, default)` reads config values; `model_load(name)` fetches a tensor (routing to the right shard); `model_close()` releases it.

```
function main(): i64 {
    // A real program points model_open() at a downloaded HF directory. This
    // builds a tiny two-shard one inline so the example is self-contained.
    mkdir("/tmp/abe_ml8_dir");
    save_safetensors("/tmp/abe_ml8_dir/s1.safetensors",
                     "model.layers.0.q_proj.weight", full(2, 3, 0.5));
    save_safetensors("/tmp/abe_ml8_dir/s2.safetensors",
                     "model.layers.0.k_proj.bias", full(4, 1, 0.25));
    write_text("/tmp/abe_ml8_dir/model.safetensors.index.json",
        "{\"weight_map\":
{\"model.layers.0.q_proj.weight\":\"s1.safetensors\",\"model.layers.0.k_proj.bias\":
\"s2.safetensors\"}}");
    write_text("/tmp/abe_ml8_dir/config.json",
        "{\"hidden_size\":512,\"num_hidden_layers\":4}");
```

```
    let r: i64 = 0;
    if (model_open("/tmp/abe_ml8_dir") != 0) { r = 1; }
    if (model_config_int("hidden_size", -1) != 512) { r = 2; }
    if (model_config_int("missing", 99) != 99) { r = 3; }   // default when absent
    // Two tensors that live on different shards — proves shard routing.
    if (tensor_sum(model_load("model.layers.0.q_proj.weight")) != 3.0) { r = 4; }
    if (tensor_sum(model_load("model.layers.0.k_proj.bias")) != 1.0) { r = 5; }
    model_close();
    return r;
}
```

A `model` (or `layer`) loads its weights by calling `model_load("<name>")` in each parameter's initializer — combine this with the architecture from Chapters 3–4 and you have a runnable LLM. Pointed at a real TinyLlama / Llama / Qwen directory, the same `model_load("model.layers.0.q_proj.weight")` names resolve the actual weights.

8.3 Stacking N blocks with `model_load_at`

A transformer has many identical blocks whose weights differ only by a layer index in the name (`layer_0_w`, `layer_1_w`, …). In an array submodule (Chapter 3), each child knows its position via the `__index__` identifier, and `model_load_at(prefix, __index__, suffix)` builds the per-index weight name at construction time:

```
layer Indexed {
    // __index__ is this layer's position in its array; model_load_at builds the
    // weight name "layer_<i>_w" so each stacked block loads its own weight.
    param weight: Tensor<f64, 2, 1> = model_load_at("layer_", __index__, "_w");
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> {
        return x + this.weight;
    }
}
model Stack {
    blocks: [3] Indexed;
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> {
        return
this.blocks[2].forward(this.blocks[1].forward(this.blocks[0].forward(x)));
    }
}
function main(): i64 {
    mkdir("/tmp/abe_ml8_idx");
    save_safetensors("/tmp/abe_ml8_idx/w0.safetensors", "layer_0_w", full(2, 1,
1.0));
    save_safetensors("/tmp/abe_ml8_idx/w1.safetensors", "layer_1_w", full(2, 1,
2.0));
    save_safetensors("/tmp/abe_ml8_idx/w2.safetensors", "layer_2_w", full(2, 1,
3.0));
    write_text("/tmp/abe_ml8_idx/model.safetensors.index.json",
        "{\"weight_map\":
{\"layer_0_w\":\"w0.safetensors\",\"layer_1_w\":\"w1.safetensors\",\"layer_2_w\":\"w
2.safetensors\"}}");
    write_text("/tmp/abe_ml8_idx/config.json", "{}");
```

```
    model_open("/tmp/abe_ml8_idx");
    let s = new Stack();
    let y = s.forward(zeros(2, 1));        // 0 + w0 + w1 + w2 = (1+2+3) per element
    model_close();
    if (tensor_sum(y) != 12.0) return 1;   // (1+2+3) over 2 elements
    return 0;
}
```

This is how an N-layer model loads N blocks' worth of distinct weights from one directory without hand-writing each name.

Function	Purpose
`save_safetensors(path, name, t)`	write a tensor to a `.safetensors` file
`load_safetensors(path, name)`	read a tensor from a file
`model_open(dir)`	open a model directory (returns 0 on success)
`model_config_int(key, default)`	read an `i64` from `config.json`
`model_count()`	number of tensors in the index
`model_load(name)`	load a tensor by name (shard-routed)
`model_load_at(prefix, i, suffix)`	load `prefix + i + suffix` (per-index weights)
`__index__`	a layer's position within its `[N] T` array
`model_close()`	release the open model

---

9. Training

Abe trains models natively: the `train` statement drives a full loop — forward, backward (reverse-mode autograd), and an optimizer step — over a dataloader, with callbacks for logging and periodic evaluation. This chapter covers training a model from scratch and parameter-efficient fine-tuning with LoRA.

9.1 The training loop

`train model on dataloader { … }` runs the loop. Inside the block you set `epochs`/`batchSize`, choose an `optimizer` (AdamW or SGD with its hyperparameters), optionally `gradientClip`, and attach **callbacks** with `on <event>(args) { … }` — `step`, `epochEnd`, `trainEnd`, and `validation`. An `eval { … }` block runs periodic validation on a held-out dataloader. After training, `export` writes the weights to safetensors.

The dataset and dataloader are runtime helpers, declared with `extern` (Level 1, Chapter 8) and used here to feed the loop:

```
extern function abe_dataset_new(): ptr;
extern function abe_dataset_demo_tokens(ds: ptr, mod_v: i64, count: i64): void;
extern function abe_dataloader_new(ds: ptr, b: i64, s: i64, shuf: bool, seed: i64):
ptr;
```

```
layer Embed<V, D> {
    param table: Tensor<f64, V, D>;
    forward(ids: Tensor<f64>): Tensor<f64> { return embedding_lookup(this.table,
ids); }
}
layer Proj<D, V> {
    param weight: Tensor<f64, D, V>;
    forward(x: Tensor<f64>): Tensor<f64> { return matmul(x, this.weight); }
}
model Tiny<V, D> {
    embed: Embed<V, D>;
    proj:  Proj<D, V>;
    forward(ids: Tensor<f64>): Tensor<f64> { return
this.proj.forward(this.embed.forward(ids)); }
}
function main(): i64 {
    let m = new Tiny<8, 4>();
    // A tiny demo dataset + dataloader (runtime helpers via extern).
    const ds = abe_dataset_new();
    abe_dataset_demo_tokens(ds, 8, 64);
    const dl = abe_dataloader_new(ds, 2, 4, false, 0);
    ai {
        train m on dl {
            epochs:    2;
            batchSize: 2;
            optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
            gradientClip: { maxNorm: 1.0 };
            on step(s, loss) if s % 4 === 0 { console.log("step", s); }
            on trainEnd(m) { console.log("done"); }
        }
        export m { format: safetensors; path: "/tmp/abe_ml9_train.safetensors"; };
    }
    console.log("trained");
    return 0;
}
```

The loop wires up automatically: the autograd tape records each op in `forward`, the backward pass computes gradients, `gradientClip` bounds them, and the optimizer updates every registered parameter. Autograd covers the transformer ops from Chapter 4 (matmul, add, the activations, softmax, RMSNorm, attention, cross-entropy), so real blocks are trainable — not just this toy.

> `train` lives inside an `ai { }` block. The callback events are `step(s, loss)`, `epochEnd(e, metrics)`, `trainEnd(metrics)`, and `validation(s, metrics)`; an `eval { data: …; interval: …; metrics: […]; }` block configures the periodic validation that fires the `validation` callback.

9.2 LoRA fine-tuning

LoRA (Low-Rank Adaptation) fine-tunes a large model by freezing its pretrained weights and training only small adapter matrices — far fewer parameters. You write the adapter arithmetic in `forward`; the `@lora(...)` decorator on the `train` statement tells the optimizer to **freeze the `weight` of each target layer and train only the adapters**:

```
extern function abe_dataset_new(): ptr;
extern function abe_dataset_demo_tokens(ds: ptr, mod_v: i64, count: i64): void;
extern function abe_dataloader_new(ds: ptr, b: i64, s: i64, sh: bool, seed: i64):
ptr;

layer Embed<V, D> {
    param table: Tensor<f64, V, D>;
    forward(ids: Tensor<f64>): Tensor<f64> { return embedding_lookup(this.table,
ids); }
}
layer LoraLinear<In, Out, R> {
    param weight: Tensor<f64, In, Out>;   // frozen base (excluded by @lora)
    param a:      Tensor<f64, In, R>;      // LoRA adapter A (trained)
    param b:      Tensor<f64, R, Out>;     // LoRA adapter B (trained)
    forward(x: Tensor<f64>): Tensor<f64> {
        const base  = matmul(x, this.weight);
        const delta = matmul(matmul(x, this.a), this.b);
        return base + delta;
    }
}
model Tiny<V, D, R> {
    embed: Embed<V, D>;
    proj:  LoraLinear<D, V, R>;
    forward(ids: Tensor<f64>): Tensor<f64> { return
this.proj.forward(this.embed.forward(ids)); }
}
function main(): i64 {
    let m = new Tiny<8, 4, 2>();
    const ds = abe_dataset_new();
    abe_dataset_demo_tokens(ds, 8, 64);
    const dl = abe_dataloader_new(ds, 2, 4, false, 0);
    ai {
        // @lora freezes each target layer's `weight` and trains only the adapters.
        @lora(rank: 2, alpha: 4, dropout: 0.0, targetLayers: ["proj"])
        train m on dl {
            epochs: 1;
            optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
        }
    }
    console.log("lora-ok");
    return 0;
}
```

The forward arithmetic (`base + delta`) is yours; `@lora` only controls **which parameters receive gradients** — the `weight` slots in `targetLayers` are frozen, the adapters `a`/`b` are trained.

Form	Purpose
`train m on dl { … }`	run the training loop
`optimizer: AdamW { lr; eps; weightDecay; }` /	choose the optimizer

Form	Purpose
`SGD { … }`	
`gradientClip: { maxNorm: … }`	clip gradient norm
`on step(s, loss) if … { … }`	per-step callback (optional guard)
`on epochEnd / trainEnd / validation(…) { … }`	lifecycle callbacks
`eval { data; interval; metrics; }`	periodic validation config
`export m { format: safetensors; path: …; }`	save trained weights
`@lora(rank, alpha, dropout, targetLayers)`	freeze base, train adapters

> **Scope.** Training is CPU `f64`, with reverse-mode autograd over the transformer op set above and the AdamW/SGD optimizers. Distributed training and mixed-precision (`@amp`) are not yet available.

---

10. Quantization

A trained model's weights are `f64` — 8 bytes per number. **Quantization** shrinks them by storing each weight in far fewer bits plus a shared scale, which is what lets a large model fit in a fraction of the memory. Abe quantizes **weights** (the dominant cost) and **dequantizes on the fly** inside `matmul`, so the math stays `f64` and the rest of your model code is unchanged.

10.1 Quantizing a model

`quantize(model, { bits: 8 })` (or `bits: 4`) converts every parameter to block-quantized form **in place** — the `f64` data is freed and replaced by a compact int8 / 4-bit payload with one scale per block of 32. A `forward` afterwards transparently dequantizes; for weights whose values are uniform, the result is unchanged:

```
layer Linear {
    param weight: Tensor<f64, 64, 64> = full(64, 64, 0.5);
    forward(x: Tensor<f64, 64, 1>): Tensor<f64, 64, 1> { return matmul(this.weight,
x); }
}
layer Weights {                          // a peek layer, just to measure storage
    param w: Tensor<f64, 64, 64> = full(64, 64, 0.5);
    forward(): Tensor<f64, 64, 64> { return this.w; }
}
function main(): i64 {
    // Correctness: a quantized layer's forward is unchanged (uniform = lossless).
    let net = new Linear();
    let x: Tensor<f64, 64, 1> = ones(64, 1);
    let before = tensor_sum(net.forward(x));         // 64 rows * (64*0.5) = 2048
    quantize(net, { bits: 8 });
    if (tensor_sum(net.forward(x)) != before) return 1;

    // Storage for a 64x64 weight: f64 -> Q8_0 -> Q4_0.
    let m8 = new Weights();
```

```
    if (tensor_nbytes(m8.forward()) != 32768) return 2;   // f64
    quantize(m8, { bits: 8 });
    if (tensor_nbytes(m8.forward()) != 5120) return 3;    // Q8_0 (~6.4x)

    let m4 = new Weights();
    quantize(m4, { bits: 4 });
    if (tensor_nbytes(m4.forward()) != 3072) return 4;    // Q4_0 (~10.7x)
    return 0;
}
```

`tensor_nbytes(t)` reports a tensor's storage. The two formats:

Format	`bits`	Storage	Reduction
`f64` (default)	—	8 B/elem	1×
Q8_0 (8-bit)	`8`	~1.25 B/elem	~6.4×
Q4_0 (4-bit)	`4`	~0.75 B/elem	~10.7×

Fewer bits means less memory but more rounding error — 8-bit is near-lossless, 4-bit trades accuracy for the bigger saving.

10.2 Saving and loading quantized weights

Quantized weights round-trip through disk **compact** — `save_safetensors` writes the int8/4-bit payload (not `f64`), and `load_safetensors` reads it back without re-expanding, so a model loaded from disk keeps its small footprint:

```
layer Src {
    param w: Tensor<f64, 64, 64> = full(64, 64, 0.5);
    forward(): Tensor<f64, 64, 64> { return this.w; }
}
layer Dst {
    param w: Tensor<f64, 64, 64> = load_safetensors("/tmp/abe_ml10.st", "w");
    forward(x: Tensor<f64, 64, 1>): Tensor<f64, 64, 1> { return matmul(this.w, x); }
}
function main(): i64 {
    let s = new Src();
    quantize(s, { bits: 8 });
    save_safetensors("/tmp/abe_ml10.st", "w", s.forward());   // persist the
quantized weight
    let d = new Dst();                                         // loads it back,
still compact
    if (tensor_sum(d.forward(ones(64, 1))) != 2048.0) return 1;
    return 0;
}
```

`export model { format: safetensors; path: … }` (Chapter 9) likewise writes a quantized model's weights compactly when the model has been quantized.

Function	Purpose	
`quantize(model, { bits: 8 \ | 4 })`	quantize every weight in place (Q8_0	

Function	Purpose	
		/ Q4_0)
`tensor_nbytes(t)`	a tensor's storage in bytes (`i64`)	
`save_safetensors` / `load_safetensors`	persist / load weights, quantized or `f64`	

> **Scope.** Quantization is **weight-only** and for **inference** — training stays `f64`. It is CPU, and loads/saves Abe's own safetensors format; loading off-the-shelf GGUF models, 4-bit super-block formats (Q4_K), and integer matmul for speed are not yet available (see the appendix).

---

11. Mixed precision

Quantization (Chapter 10) shrinks weights to a few bits. **Mixed precision** is the gentler, standard middle ground: store weights as **16-bit floats** instead of `f64`. Two formats are available — **bf16** (a 7-bit mantissa with `f32`'s full exponent range) and **f16** (IEEE-754 half: a 10-bit mantissa but a narrower range). Both are **a quarter** the size of `f64`. As with quantization, Abe widens a 16-bit weight back to `f64` **on the fly** inside `matmul`, so the kernels and the rest of your code are unchanged.

11.1 Casting tensors and models

`to_bf16` / `to_f16` convert a tensor **in place** to 16-bit storage; `to_f64` widens it back. Applied to a **model**, they walk every parameter at once:

```
layer Net {
    param w: Tensor<f64, 4, 64> = full(4, 64, 0.5);
    forward(x: Tensor<f64, 64, 1>): Tensor<f64, 64, 1> { return matmul(this.w, x); }
}
function main(): i64 {
    // Tensor-level: 256-elem weight, f64 2048 B -> bf16 512 B (4x). 0.5 is exact.
    let a = full(4, 64, 0.5);
    if (tensor_nbytes(a) != 2048) return 1;
    to_bf16(a);
    if (tensor_nbytes(a) != 512) return 2;
    if (tensor_sum(matmul(a, ones(64, 1))) != 128.0) return 3;

    // f16 is the same footprint; to_f64 widens back.
    let b = full(4, 64, 0.5);
    to_f16(b);
    if (tensor_nbytes(b) != 512) return 4;
    to_f64(b);
    if (tensor_nbytes(b) != 2048) return 5;

    // Model-level: cast every param at once, then widen back.
    let n = new Net();
    to_bf16(n);
    if (tensor_sum(n.forward(ones(64, 1))) != 128.0) return 6;
    to_f64(n);
    if (tensor_sum(n.forward(ones(64, 1))) != 128.0) return 7;
```

```
    return 0;
}
```

bf16 and f16 trade precision differently: for `0.1`, bf16 rounds to `0.100098` and f16 to `0.099976` — f16's extra mantissa bits make it finer, while bf16's wider exponent range makes it safer against overflow on large values. `0.5` (and any exact power-of-two combination) round-trips losslessly in both, which is why the checks above are exact.

Format	Cast	Storage	vs `f64`
`f64` (default)	`to_f64`	8 B/elem	1×
bf16	`to_bf16`	2 B/elem	4×
f16	`to_f16`	2 B/elem	4×

11.2 Loading a 16-bit checkpoint compact

A real checkpoint is often stored in bf16/f16 already. By default the loader **expands** such weights to `f64` on load (4× the memory). `load_compact(true)` tells it to keep them in their narrow form — `matmul` widens on the fly — so a bf16 model loads at a quarter of the peak memory. The on-disk format is standard safetensors, so files interoperate with other tools:

```
layer Src {
    param w: Tensor<f64, 64, 64> = full(64, 64, 0.5);
    forward(): Tensor<f64, 64, 64> { return this.w; }
}
layer Dst {
    param w: Tensor<f64, 64, 64> = load_safetensors("/tmp/abe_ml11.st", "w");
    forward(x: Tensor<f64, 64, 1>): Tensor<f64, 64, 1> { return matmul(this.w, x); }
}
layer Peek {
    param w: Tensor<f64, 64, 64> = load_safetensors("/tmp/abe_ml11.st", "w");
    forward(): Tensor<f64, 64, 64> { return this.w; }
}
function main(): i64 {
    let s = new Src();
    to_bf16(s);
    save_safetensors("/tmp/abe_ml11.st", "w", s.forward());   // standard BF16
safetensors
    load_compact(true);                                       // keep it compact on
reload
    let d = new Dst();
    if (tensor_sum(d.forward(ones(64, 1))) != 2048.0) return 1;
    let p = new Peek();
    if (tensor_nbytes(p.forward()) != 8192) return 2;         // compact bf16 (4096
* 2)
    return 0;
}
```

`load_compact` is **opt-in** because a compact weight is **matmul-only**: load the big projection weights compact and leave small `rmsnorm` / embedding weights on the default `f64` path (or widen them with `to_f64`).

11.3 bf16 training with `@amp`

Decorate a `train` statement with `@amp` to run a **bf16-numeric forward**: the forward's matmuls round their operands through bf16, while the master weights and the whole backward pass stay `f64` — the standard automatic-mixed-precision recipe. This reproduces the *numerics* of bf16 training so you can check that a model still converges under bf16 rounding:

```
ai {
    @amp(dtype: bf16)
    train m on dl {
        epochs: 2;
        optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
        on trainEnd(m) { console.log("amp train ok"); }
    }
}
```

`@amp` defaults to `bf16`; `@amp(dtype: fp16)` is rejected, because fp16 needs dynamic loss scaling that the `f64` backward can't meaningfully provide.

Tool	Purpose	
`to_bf16(t \	model) `/` to_f16(...)`	cast to 16-bit storage in place (4× smaller)
`to_f64(t \	model)`	widen back to `f64`
`load_compact(on)`	keep F16/BF16 weights compact on load (matmul-only)	
`@amp` on `train`	bf16-numeric forward, `f64` master weights + backward	

> **Scope.** bf16/f16 are **compact storage** for **inference** weights (matmul-only, like quantized tensors) — they are not yet trainable directly, and non-`matmul` kernels still need `f64`. `@amp` training reproduces bf16 *numerics* but keeps `f64` storage, so it is not yet a memory or throughput win — true narrow-storage training (with `f32` master weights) is a later step.

---

12. Running on the GPU

Everything so far runs on the CPU. Abe also has a **CUDA backend**: the same program, unchanged, can run its tensor ops on an NVIDIA GPU. Tensors stay **device-resident** (matmul uses cuBLAS; element-wise ops, softmax, and paged attention are CUDA kernels), so data isn't shuffled host ↔ device between ops.

Two steps — link it, then select it:

1. **Compile with `--gpu`** so the CUDA backend is linked in:

```
abec --gpu mymodel.abe -o mymodel
```

1. **Select the backend at run time** with `ABE_BACKEND`:

```
 ABE_BACKEND=cuda ./mymodel # use the GPU (fall back to CPU + warn if none)
ABE_BACKEND=cpu ./mymodel # force CPU even if a GPU is present ./mymodel # auto: GPU if one is visible, else CPU
```

The program itself doesn't change. This one runs identically on either backend — the values are exact f64 fractions, so CPU and CUDA agree bit-for-bit:

```
function main(): i64 {
    let a = full(8, 16, 0.25);
    let b = full(16, 8, 0.5);
    let c = matmul(a, b);          // (8,8); each entry = 16 * 0.25 * 0.5 = 2.0
    let r = relu(c);               // 2.0 (unchanged, all positive)
    let s = tensor_sum(r);         // 8 * 8 * 2.0 = 128
    if (s == 128.0) { console.log("gpu parity ok"); }
    return 0;
}
```

Setting	Effect
`abec --gpu …`	link the CUDA backend (`libabe_gpu.a`); needs `nvcc` at build time
`ABE_BACKEND=cuda`	run on the GPU (warns + falls back to CPU if none is visible)
`ABE_BACKEND=cpu`	force the CPU backend
`ABE_BACKEND=auto` *(default)*	GPU if a CUDA device is visible, else CPU

Scope. The GPU path is **f64** and built for **correctness** — it matches the CPU backend op-for-op (the test suite asserts CPU/CUDA parity), but the kernels are **not throughput-tuned** yet (no autotuning; whole-tensor softmax is single-block; transpose is 2-D only; non-last-axis softmax and RNG fall back to the host). Treat it as a correct reference GPU backend, not a peak-FLOPs one.

---

13. Gradients and einsum

Chapter 9 trained models with the `train` statement, which drives the autograd engine for you. Two expressions let you reach the same machinery directly.

13.1 `gradient(loss, params)`

`gradient(loss, params)` returns the reverse-mode derivative of a **scalar** `loss` with respect to a single tensor `params`. The loss is built from the param with ordinary tensor ops; `gradient` records them, runs the backward pass, and hands back the gradient (same shape as `params`):

```
function main(): i64 {
    let w: Tensor<f64, 1, 4> = full(1, 4, 0.0);
    let x: Tensor<f64, 4, 1> = full(4, 1, 3.0);
    let g = gradient(matmul(w, x), w);          // loss = w·x (1x1); d(loss)/dw = x
    if (tensor_sum(g) == 12.0) { console.log("gradient ok"); }   // 4 * 3
    console.log("grad numel:", tensor_numel(g));                 // 4
```

```
    return 0;
}
```

The loss must be a scalar (a size-1 tensor), exactly as in a training step.

13.2 `einsum`

`einsum("fmt", operands…)` contracts tensors by an index spec — the same notation NumPy/PyTorch use. The format is explicit (`inputs -> output`), labels are lowercase `a-z`, and up to 8 operands are supported:

```
function main(): i64 {
    let a = full(2, 3, 2.0);
    let b = full(3, 4, 0.5);
    let c = einsum("ij,jk->ik", a, b);   // matmul: (2,4), each = 3 * 2.0 * 0.5 =
3.0
    let t = einsum("ij->ji", a);         // transpose: (3,2)
    let s = einsum("ij->", a);           // sum-all: 12
    if (tensor_sum(c) == 24.0 && tensor_numel(t) == 6 && tensor_sum(s) == 12.0) {
        console.log("einsum ok");
    }
    return 0;
}
```

Handy patterns: `"ij,jk->ik"` (matmul), `"ij->ji"` (transpose), `"ii->i"` (diagonal), `"ij->"` (sum-all), `"bij,bjk->bik"` (batched matmul). Operands must be dense `f64` tensors — widen a compact bf16/quantized weight with `to_f64` first.

Expression	Purpose
`gradient(loss, params)`	reverse-mode d(loss)/d(params), scalar loss, one tensor param
`einsum("fmt", a, b, …)`	Einstein-summation contraction (1–8 dense f64 operands)
`jacobian(fn, x)`	reverse-mode Jacobian matrix of a top-level `fn: (Tensor)->Tensor`
`valueAndGrad(fn)(x)`	`[value, grad]` for a top-level scalar `fn`; `let (v, g) = …`

```
function f(x: Tensor<f64, 1, 4>): Tensor<f64, 1, 1> { return matmul(x, full(4, 1,
2.0)); }
function main(): i64 {
    let x: Tensor<f64, 1, 4> = full(1, 4, 1.0);
    let (v, g) = valueAndGrad(f)(x);   // v = sum(2x) = 8; g = [2,2,2,2]
    if (tensor_sum(v) == 8.0 && tensor_sum(g) == 8.0) { console.log("ok"); }
    return 0;
}
```

Scope. `gradient` differentiates a scalar loss w.r.t. one tensor. `jacobian(fn, x)` and `valueAndGrad(fn)(x)` take a top-level single-tensor-arg function (closures/arrows and partial

> application `valueAndGrad(fn)` aren't supported). `valueAndGrad` returns a 2-element list — destructure it with `let (v, g) = …` (positional `[a,b]` / `(a,b)` destructuring binding works generally now) or index `r[0]` / `r[1]`.

---

14. Convolution

For vision and signal models, `conv2d` and `conv1d` provide direct (f64) convolution — really cross-correlation, the ML convention — with zero padding and unit dilation:

- `conv2d(input, weight, stride, pad)` — `input` is `(N, Cin, H, W)` (or `(Cin, H, W)`), `weight` is `(Cout, Cin, KH, KW)`; output is `(N, Cout, Hout, Wout)` with `Hout = (H + 2·pad − KH)/stride + 1`.
- `conv1d(input, weight, stride, pad)` — the 1-D analogue: `(N, Cin, L)` with `(Cout, Cin, K)` → `(N, Cout, Lout)`.

```
function main(): i64 {
    // 3x3 input of ones, 2x2 kernel of ones, stride 1, pad 0 -> 2x2, each = 4.
    let x = full(1, 1, 3, 3, 1.0);
    let w = full(1, 1, 2, 2, 1.0);
    let y = conv2d(x, w, 1, 0);

    let xa = full(1, 1, 5, 1.0);
    let wa = full(1, 1, 3, 1.0);
    let ya = conv1d(xa, wa, 1, 0);       // length 3, each = 3

    if (tensor_sum(y) == 16.0 && tensor_numel(y) == 4 &&
        tensor_sum(ya) == 9.0 && tensor_numel(ya) == 3) {
        console.log("conv ok");
    }
    return 0;
}
```

Builtin	Shapes
`conv2d(input, weight, stride, pad)`	`(N,Cin,H,W)` ⊛ `(Cout,Cin,KH,KW)` → `(N,Cout,Hout,Wout)`
`conv1d(input, weight, stride, pad)`	`(N,Cin,L)` ⊛ `(Cout,Cin,K)` → `(N,Cout,Lout)`

> **Scope.** Direct f64 convolution, no bias term (add one with a broadcast), unit dilation, no groups. Fine for small CNNs and signal filters; not tuned for large-image throughput.

---

15. Object-oriented layers and models

A `layer` or `model` is not limited to a single `forward`. It can declare **custom methods**, a custom `**init()` **constructor body, and (for models) a** `config {}`** block of named hyperparameters — so a building block carries its own behaviour, not just its parameters.

15.1 Custom methods

A method other than `forward` lowers to `<Layer>__<method>(this, args…)` and dispatches the same way — callable from outside the instance and from `forward` via `this.<method>(…)`:

```
layer Block {
    param w: Tensor<f64, 4, 4> = full(4, 4, 0.5);
    project(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {   // a custom method
        return matmul(this.w, x);
    }
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {
        return this.project(x);            // call a custom method internally
    }
}
function main(): i64 {
    let b = new Block();
    let y = b.project(ones(4, 1));         // external dispatch: 4 * (4*0.5) = 8
    let f = b.forward(ones(4, 1));         // forward delegates to project: 8
    if (tensor_sum(y) == 8.0 && tensor_sum(f) == 8.0) { console.log("layer method
ok"); }
    return 0;
}
```

15.2 `init()` — a constructor body

Beyond `param = expr` field initializers, an `init()` block runs at construction **after** the params are allocated, with `this` in scope. It can read params, call methods, and assign params (`this.field = expr`):

```
layer Block {
    param w: Tensor<f64, 2, 2> = full(2, 2, 0.0);
    init() {
        this.w = full(2, 2, 3.0);          // compute & assign a param in init
    }
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> { return matmul(this.w, x); }
}
function main(): i64 {
    let b = new Block();                    // init() runs here
    if (tensor_sum(b.forward(ones(2, 1))) == 12.0) { console.log("init ok"); }  //
2*(2*3)
    return 0;
}
```

15.3 Methods and `init()` on models

Models get the same surface — custom methods and an `init()` body, with internal `this.method(…)` dispatch:

```
model Net {
    shared param w: Tensor<f64, 4, 4> = full(4, 4, 0.0);
    init() { this.w = full(4, 4, 1.0); }                                    //
set in init
```

```
    score(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> { return matmul(this.w, x); }  //
custom method
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> { return this.score(x); }    //
internal dispatch
}
function main(): i64 {
    let n = new Net();
    // init set w=1.0 (not the 0.0 default); forward via the custom method:
4*(4*1.0) = 16
    if (tensor_sum(n.forward(ones(4, 1))) == 16.0) { console.log("model oo ok"); }
    return 0;
}
```

(`shared param` is covered in Chapter 16.)

15.4 `config {}` — model hyperparameters

A model `config {}` block declares named constants that are **in scope in the model's methods** (`forward`, custom methods, `init`), so the body reads them directly instead of threading them through every call:

```
model Hyper {
    config {
        layers: i64 = 12;
        scale:  f64 = 0.5;
    }
    forward(): i64 {
        return layers;          // a config value, in scope in the model body
    }
}
function main(): i64 {
    let h = new Hyper();
    if (h.forward() == 12) { console.log("config ok"); }
    return 0;
}
```

15.5 `noGrad` / `inferenceMode`

Inside an `ai {}` block, a `noGrad { … }` or `inferenceMode { … }` block disables gradient tracking for its body and restores the previous state on exit — the way to run pure inference without paying for the autograd tape:

```
layer Net {
    param w: Tensor<f64, 4, 4> = full(4, 4, 0.5);
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> { return matmul(this.w, x); }
}
function main(): i64 {
    let n = new Net();
    let x: Tensor<f64, 4, 1> = ones(4, 1);
    ai {
        inferenceMode {                     // gradient tracking off inside the
block
            let y = n.forward(x);           // 4 rows * (4*0.5) = 8
```

```
            if (tensor_sum(y) == 8.0) { console.log("inference mode ok"); }
        }
        noGrad {                              // same, restored on exit
            let z = n.forward(x);
            if (tensor_sum(z) == 8.0) { console.log("no grad ok"); }
        }
    }
    return 0;
}
```

---

16. Weight tying

Tying weights — making two places in a network share one tensor — is standard in transformers (tied input embedding / output projection). Abe expresses it two ways.

16.1 `shared param` — one model-owned tensor

A model `shared param w: …` is allocated **once**, reachable as `this.w` everywhere in the model, and registered with the optimizer once. Using it in more than one place ties those uses to the same tensor:

```
model Tied {
    shared param w: Tensor<f64, 4, 4> = full(4, 4, 0.5);
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> {
        return matmul(this.w, matmul(this.w, x));   // the same tied tensor, used
twice
    }
}
function main(): i64 {
    let m = new Tied();
    // w @ (w @ 1): inner row = 4*0.5 = 2; outer row = 4*0.5*2 = 4 -> sum = 16
    if (tensor_sum(m.forward(ones(4, 1))) == 16.0) { console.log("tie ok"); }
    return 0;
}
```

16.2 Cross-submodule tying

`this.<submodule>.<param>` reads and writes a submodule's parameter, so you can tie one submodule's weight to another — assign one to the other in `init()` and they share the same tensor afterwards:

```
layer Inner {
    param w: Tensor<f64, 2, 2> = full(2, 2, 0.5);
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> { return matmul(this.w, x); }
}
model Net {
    a: Inner;
    b: Inner;
    init() {
        this.a.w = full(2, 2, 3.0);    // set a's weight (nested write)
```

```
        this.b.w = this.a.w;           // tie b to a (nested read + write -> shared
tensor)
    }
    forward(x: Tensor<f64, 2, 1>): Tensor<f64, 2, 1> { return this.b.forward(x); }
}
function main(): i64 {
    let n = new Net();
    // b.w is tied to a.w (=3): 2 rows * (2 * 3) = 12. Untied (default 0.5) would be
2.
    if (tensor_sum(n.forward(ones(2, 1))) == 12.0) { console.log("cross tie ok"); }
    return 0;
}
```

> **Training caveat.** A tied weight is registered once **per slot**, so a shared weight that is trained may have its update counted more than once. This is harmless for inference; for training, prefer the single `shared param` form (16.1), which registers exactly once.

---

17. Mixture of Experts

Abe has the primitives for a Mixture-of-Experts (MoE) layer: a learned router gate, top-k gating math, and the Switch-Transformer load-balancing loss. The expert **dispatch/combine** itself is ordinary `forward` code (loop the expert submodules, weight each by its gate column) — it can't be a runtime builtin, because that would have to call back into your expert layers.

17.1 A `router` gate

`router name: Tensor<…> = …;` declares a learned gating weight inside a layer. It is a trainable param-shaped slot: allocated in the constructor, reachable as `this.<name>`, and registered with the optimizer. Experts are ordinary submodules alongside it:

```
layer Expert {
    param w: Tensor<f64, 4, 4> = full(4, 4, 0.5);
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 4, 1> { return matmul(this.w, x); }
}
layer MoE {
    router gate: Tensor<f64, 2, 4> = full(2, 4, 0.25);   // learned gate over D=4
for 2 experts
    param e0: Expert;
    param e1: Expert;
    forward(x: Tensor<f64, 4, 1>): Tensor<f64, 2, 1> {
        let _y0 = this.e0.forward(x);          // experts coexist with the router
        let _y1 = this.e1.forward(x);
        return softmax(matmul(this.gate, x));  // gate logits -> gate weights
    }
}
function main(): i64 {
    let m = new MoE();
    // gate=0.25, x=ones(4,1) -> logits [1,1] -> softmax [0.5,0.5] -> sum 1.0
    if (tensor_sum(m.forward(ones(4, 1))) == 1.0) { console.log("moe router ok"); }
```

```
    return 0;
}
```

17.2 Gating builtins: `xavier_uniform`, `topk_gate`

`xavier_uniform(rows, cols)` is a Glorot-uniform initializer (the standard init for a router gate), shaped like `randn`. `topk_gate(scores, k)` keeps the `k` largest scores per row (over the last axis = experts), zeros the rest, and renormalizes the kept weights to sum to 1 — the top-k gating math:

```
function main(): i64 {
    // Glorot init: (3,4) -> 12 values in [-sqrt(6/7), sqrt(6/7)].
    let gate = xavier_uniform(3, 4);
    // Top-k gating over 4 experts (last axis): keep top-2, renormalize.
    let scores = full(1, 4, 1.0);
    let g = topk_gate(scores, 2);              // two equal winners -> 0.5, 0.5
    if (tensor_numel(gate) == 12 &&
        tensor_sum(g) == 1.0 && tensor_numel(g) == 4) {
        console.log("moe dispatch ok");
    }
    return 0;
}
```

17.3 Load-balancing auxiliary loss

`moe_balance_loss(gate_probs, k)` returns the scalar $E \cdot \Sigma_i f_i \cdot P_i$ (Switch Transformer), where P_i is the mean router probability for expert i and f_i is the fraction of tokens that route expert i into their top-k. It is minimized by balanced routing; add it (times a small coefficient) to the task loss to discourage expert collapse. It is differentiable through P_i, so backprop nudges the gate toward balance:

```
function main(): i64 {
    // 1 token, 4 experts, uniform probs; top-2 -> experts 0,1 (tie-break).
    // P=[.25,.25,.25,.25], f=[1,1,0,0]  ->  aux = 4*(.25+.25) = 2.0
    let probs = full(1, 4, 0.25);
    let aux = moe_balance_loss(probs, 2);
    // gradient flows through P_i: d(aux)/d(probs[0,i]) = (E/N)*f_i = 4*f_i ->
[4,4,0,0], sum 8
    let p2 = full(1, 4, 0.25);
    let g = gradient(moe_balance_loss(p2, 2), p2);
    if (tensor_sum(aux) == 2.0 && tensor_sum(g) == 8.0) { console.log("moe balance
ok"); }
    return 0;
}
```

Builtin	Purpose
`router name: T = …`	a learned, trainable gating weight in a layer
`xavier_uniform(rows, cols)`	Glorot-uniform initializer (router-gate init)
`topk_gate(scores, k)`	top-k renormalized gating weights over the last axis
`moe_balance_loss(probs, k)`	Switch-Transformer load-balancing aux loss (differentiable)

---

18. Custom gradients

A layer that declares a `backward(gradOut): gradIn` method gets a **custom gradient** (PyTorch `Function.backward` style): its `forward` runs untaped and the autograd engine calls `backward` to turn the output gradient into the input gradient. The clean use is a custom-gradient op or activation — it is param-less (parameter gradients inside such a layer are not auto-computed).

The example below deliberately makes `backward` return `3·gradOut` while `forward` computes `2·x`, so the result (3, not 2) proves the custom backward is the path actually taken:

```
layer Op {
    forward(x: Tensor<f64, 1, 1>): Tensor<f64, 1, 1> { return x + x; }        // 2x
    backward(g: Tensor<f64, 1, 1>): Tensor<f64, 1, 1> { return g + g + g; }   //
custom: 3*g
}
function main(): i64 {
    let op = new Op();
    let x: Tensor<f64, 1, 1> = full(1, 1, 5.0);
    let grad = gradient(op.forward(x), x);     // scalar loss; custom backward -> 3
    if (tensor_sum(grad) == 3.0) { console.log("custom backward ok"); }
    return 0;
}
```

---

19. Knowledge distillation

`distill student on <data> from teacher { … }` trains a small **student** to imitate a larger **teacher**. Each step runs the frozen teacher forward (untaped), the student forward (taped), and a KD loss that blends a soft teacher-matching term (at `temperature` T) with the hard cross-entropy (weight `1 − alpha`); only the student is optimized. The block takes the same `epochs` / `optimizer` / event hooks as `train` (Chapter 9), plus `temperature` and `alpha`:

```
extern function abe_dataset_new(): ptr;
extern function abe_dataset_demo_tokens(ds: ptr, mod_v: i64, count: i64): void;
extern function abe_dataloader_new(ds: ptr, b: i64, s: i64, shuf: bool, seed: i64):
ptr;

layer Embed<V, D> {
    param table: Tensor<f64, V, D> = randn(V, D);
    forward(ids: Tensor<f64>): Tensor<f64> { return embedding_lookup(this.table,
ids); }
}
layer Proj<D, V> {
    param weight: Tensor<f64, D, V> = randn(D, V);
    forward(x: Tensor<f64>): Tensor<f64> { return matmul(x, this.weight); }
}
model Tiny<V, D> {
```

```
    embed: Embed<V, D>;
    proj:  Proj<D, V>;
    forward(ids: Tensor<f64>): Tensor<f64> { return
this.proj.forward(this.embed.forward(ids)); }
}
function main(): i64 {
    let student = new Tiny<8, 4>();
    let teacher = new Tiny<8, 4>();
    const ds = abe_dataset_new();
    abe_dataset_demo_tokens(ds, 8, 64);
    const dl = abe_dataloader_new(ds, 2, 4, false, 0);
    ai {
        distill student on dl from teacher {
            epochs: 2;
            temperature: 2.0;
            alpha: 0.5;
            optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
            on trainEnd(m) { console.log("distill ok"); }
        }
    }
    return 0;
}
```

The soft KD loss decreases as the student approaches the teacher (measured separately at roughly 0.041 → 0.001 over a few epochs on this fixture).

---

20. Conversational AI

Abe has statement forms for assembling prompts and running a simple interactive dialog. `prompt { … }` (inside `ai {}`) joins its property values into one newline-separated string; the `dialog {}` block adds `ask` / `respond` / `confirm`:

- `prompt { system; user; … }` → one newline-joined prompt string.
- `ask user { prompt: … }` → ask the user and bind the reply text.
- `respond { message: … }` → print a reply.
- `confirm { prompt: … }` → ask a yes/no question and bind a `bool`.

```
function main(): i64 {
    ai {
        // prompt { … } joins its property values into one newline-joined string.
        let p = prompt { system: "SYS"; user: "USR"; };
        console.log("prompt:", p);          // SYS\nUSR
    }
    dialog {
        let name = ask user { prompt: "Name?"; };   // ask + bind the reply
        respond { message: name; };                  // print a reply
        let ok = confirm { prompt: "Continue?"; };   // yes/no -> bool
    }
```

```
    return 0;
}
```

`ask` / `confirm` read a line from standard input; on end-of-input they yield an empty reply / the default, so a non-interactive run does not block.

---

21. Distributed training

`@ddp(worldSize: N)` on a `train` runs a **single-process** simulation of Distributed Data Parallel: the optimizer accumulates and averages gradients over `N` consecutive micro-batches, then takes one real step — the same math DDP performs across `N` GPUs, giving an `N`× larger effective batch. It is **not** true multi-GPU/NCCL on this build, and the compiler says so with an info diagnostic rather than pretending otherwise:

```
extern function abe_dataset_new(): ptr;
extern function abe_dataset_demo_tokens(ds: ptr, mod_v: i64, count: i64): void;
extern function abe_dataloader_new(ds: ptr, b: i64, s: i64, shuf: bool, seed: i64):
ptr;

layer Embed<V, D> {
    param table: Tensor<f64, V, D> = randn(V, D);
    forward(ids: Tensor<f64>): Tensor<f64> { return embedding_lookup(this.table,
ids); }
}
layer Proj<D, V> {
    param weight: Tensor<f64, D, V> = randn(D, V);
    forward(x: Tensor<f64>): Tensor<f64> { return matmul(x, this.weight); }
}
model Tiny<V, D> {
    embed: Embed<V, D>;
    proj:  Proj<D, V>;
    forward(ids: Tensor<f64>): Tensor<f64> { return
this.proj.forward(this.embed.forward(ids)); }
}
function main(): i64 {
    let m = new Tiny<8, 4>();
    const ds = abe_dataset_new();
    abe_dataset_demo_tokens(ds, 8, 64);
    const dl = abe_dataloader_new(ds, 2, 4, false, 0);
    ai {
        @ddp(worldSize: 4)
        train m on dl {
            epochs: 2;
            optimizer: AdamW { lr: 1e-2; eps: 1e-8; weightDecay: 0.0; };
            on trainEnd(m) { console.log("ddp train ok"); }
        }
    }
    return 0;
```

```
}
```

`@fsdp` / `@tensorParallel` / `@pipeline` are accepted and acknowledged (with an info diagnostic) but have no effect on this single-process build — real parameter/tensor/pipeline sharding needs multi-GPU hardware and NCCL collectives.

---

22. ONNX interoperability

Abe can load and run a pre-trained **ONNX** model through onnxruntime:

- `onnx_load(path)` → an opaque session handle (`null` on failure).
- `onnx_run(session, x)` → the output `Tensor`.
- `onnx_free(session)` → release the session.

onnxruntime is an **optional** dependency: the runtime is built against it by pointing `ORT_HOME` at an onnxruntime install (see `runtime/Makefile` and `make -C runtime test-onnx`). Without it, `onnx_load` returns `null` and prints an honest "not available" message — so guard the handle and the same source runs either way:

```
function main(): i64 {
    let m = onnx_load("/tmp/relu_f64.onnx");
    if (m == null) {                       // runtime built without onnxruntime
        console.log("onnx ok (stub: onnxruntime not built)");
        return 0;
    }
    let y = onnx_run(m, full(4, 1, 3.0));  // Relu([3,3,3,3]) = [3,3,3,3], sum 12
    if (tensor_sum(y) == 12.0) { console.log("onnx ok"); }
    onnx_free(m);
    return 0;
}
```

The feed/fetch tensors are `f64`; the model's input/output element type must be `double` (the integration's end-to-end test uses a hand-encoded `Relu` double model, verified to produce `[0,2,0,4]` from `[-1,2,-3,4]`).

Appendix — Current limitations

Abe's ML stack runs a complete Llama-class inference and training pipeline today, but it is still maturing. So you know exactly what to expect, this release does **not** yet include:

- **Precision / hardware:** tensor math accumulates in `f64` on both backends. The **CUDA GPU backend works and matches the CPU op-for-op** (Chapter 12), but it is a correct reference — not throughput-tuned (no autotuning; a few ops fall back to the host). Mixed precision is available as **compact 16-bit storage** (bf16/f16; Chapter 11) and a **bf16-numeric `@amp` forward** for training — but storage-side bf16/f16 is inference-only (matmul widens to `f64`), and fp16 with loss scaling is not wired.

- **Quantization:** Q8_0 (8-bit) and Q4_0 (4-bit) **weight** quantization are available (Chapter 10) — quantize, run, and save/load compact. Not yet: loading off-the-shelf **GGUF** models, 4-bit super-block formats (Q4_K), and integer (int8) matmul for speed — quantization shrinks memory, not yet runtime.
- **Distributed training:** `@ddp(worldSize: N)` on a `train` runs a single-process simulation — gradients are accumulated and averaged over N micro-batches per optimizer step (the DDP math, = an N× effective batch), not true multi-GPU. `@fsdp` / `@tensorParallel` / `@pipeline` are acknowledged (with an info diagnostic) but have no effect: real cross-device sharding + NCCL collectives need multi-GPU hardware.
- **Autograd reach:** explicit autograd (`gradient`, `einsum`, `jacobian`, `valueAndGrad`; Chapter 13) covers a scalar loss / a top-level single-tensor-arg function. Partial application `valueAndGrad(fn)` and closure/arrow `fn`s aren't supported — they need first-class function values. A layer custom `backward` (Chapter 18) is a param-less custom-gradient op: parameter gradients inside such a layer are not auto-computed.
- **MoE dispatch:** the router gate, top-k gating, and load-balancing loss are builtins (Chapter 17), but the expert **dispatch/combine** is ordinary `forward` code — it can't be a builtin, which would have to call your expert layers.
- **Weight tying:** a tied weight registered **per slot** (the cross-submodule form, 16.2) may double-count its update during training; prefer a single `shared param` (16.1) when training a tied weight. (Inference is unaffected.)
- **ONNX:** onnxruntime is an **optional** dependency (Chapter 22). A default build links an honest "not available" stub (`onnx_load` returns `null`); real inference needs a runtime built against onnxruntime (`ORT_HOME`). Feed/fetch tensors are `f64` (the model's I/O element type must be `double`).
- **Other ops:** `conv2d`/`conv1d` are direct f64 (Chapter 14; no groups/bias/dilation); `reshape` always copies. `matmul` operates on 2-D tensors, which is why layer inputs use the column form `Tensor<f64, In, 1>`.

These constraints are the roadmap, not the destination.

Abe Pro ML Manual (Level 2) — complete: Chapters 1–22 (Tensors, Layers, Models, Attention, Inference, Tokenization, Serving, Loading models, Training, Quantization, Mixed precision, Running on the GPU, Gradients and einsum, Convolution, Object-oriented layers and models, Weight tying, Mixture of Experts, Custom gradients, Knowledge distillation, Conversational AI, Distributed training, ONNX interoperability). Every example is compiled and run by `tests/run_manual_ml_examples.sh` (`make test-manual-ml`).